

Exception handling in Apex



Exception handling in apex in detail with multiple examples and explain all the examples

Absolutely—let's dive **deep into exception handling in Apex**, the Salesforce programming language. I'll explain:

- ✓ What exceptions are in Apex
- ✓ How to handle them (**try**, **catch**, **finally**)
- ✓ Built-in exception types
- ✓ How to create custom exceptions
- ✓ Multiple examples with explanations

★ What is Exception Handling?

Exceptions are runtime errors that occur when something goes wrong during code execution, such as:

- **DML errors** (e.g., inserting duplicate records)
- **Null pointer errors**
- **Query errors**
- **Business rule violations**

Exception handling allows you to **gracefully catch these errors**, log them, or notify users instead of letting the code fail.

Apex provides:

- **try** block: code you *try* to run
- **catch** block: code that handles exceptions
- **finally** block: code that always runs (optional)

★ Syntax of Exception Handling

```
apex

try {
    // Code that may throw an exception
} catch (ExceptionType e) {
    // Handle the exception
} finally {
    // Code that always runs (optional)
}
```

Built-in Exception Types in Apex

Some common built-in exceptions:

Exception Class	When It Occurs
DmlException	DML operations fail (insert, update, delete)
NullPointerException	Code tries to access a null variable
QueryException	SOQL queries fail (e.g., too many rows)
MathException	Invalid math operation (e.g., divide by zero)
SObjectException	Problems with SObject data
ListException	Invalid list operations
StringException	Invalid string operations
TypeException	Invalid type casting

You can also **catch the base Exception class** to handle any exception.

Example 1: Handling a DmlException

Scenario: Insert an Account with a duplicate value (e.g., unique Name).

```

apex

try {
    Account acc = new Account(Name='Acme');
    insert acc;

    // Try inserting again with the same Name if it's unique
    Account duplicateAcc = new Account(Name='Acme');
    insert duplicateAcc;
} catch (DmlException e) {
    System.debug('DML Exception: ' + e.getMessage());
}
  
```

Explanation:

- The first insert succeeds.
- The second insert may fail (if Name is unique).
- **DmlException** is caught and logged.

Example 2: Handling a NullPointerException

Scenario: Attempt to access a property of a null object.

```

apex

try {
    Account acc = null;
    String name = acc.Name; // This line throws NullPointerException
} catch (NullPointerException e) {
  
```

```
System.debug('Null Pointer Exception: ' + e.getMessage());
}
```

✓ Explanation:

- `acc` is `null`.
- Trying to access `acc.Name` throws `NullPointerException`.
- It's caught and logged.

★ Example 3: Handling a QueryException

Scenario: Query expecting one record but getting many.

```
apex

try {
    Account acc = [SELECT Id, Name FROM Account]; // No LIMIT specified
} catch (QueryException e) {
    System.debug('Query Exception: ' + e.getMessage());
}
```

✓ Explanation:

- Query without `LIMIT` may return >1 record.
- Throws `QueryException`.
- Caught and debugged.

★ Example 4: Using finally Block

Scenario: Log after exception handling, regardless of success.

```
apex

try {
    Account acc = [SELECT Id FROM Account WHERE Name='Nonexistent' LIMIT 1];
    delete acc;
} catch (Exception e) {
    System.debug('Exception: ' + e.getMessage());
} finally {
    System.debug('Finally block executed.');
```

✓ Explanation:

- If query fails (no record found), it throws `QueryException`.
- `catch` handles it.
- `finally` always executes.

🌟 Example 5: Catching Multiple Exception Types

Scenario: Handle different exceptions differently.

```
apex

try {
    Account acc = [SELECT Id FROM Account LIMIT 1];
    acc = null;
    String name = acc.Name; // NullPointerException
} catch (NullPointerException e) {
    System.debug('Null pointer error: ' + e.getMessage());
} catch (QueryException e) {
    System.debug('Query error: ' + e.getMessage());
} catch (Exception e) {
    System.debug('Other error: ' + e.getMessage());
}
```

✅ Explanation:

- If `NullPointerException` occurs, first `catch` block handles it.
- If `QueryException` occurs, second `catch` block handles it.
- Any other exception goes to the last `catch`.

🌟 Example 6: Custom Exception Class

Scenario: Enforce a business rule—e.g., Account Name cannot be “Test”.

✅ Define Custom Exception:

```
apex

public class InvalidAccountNameException extends Exception {}
```

✅ Use it:

```
apex

try {
    Account acc = new Account(Name='Test');
    if (acc.Name == 'Test') {
        throw new InvalidAccountNameException('Account name "Test" is not allowed.');
```

```
    }
    insert acc;
} catch (InvalidAccountNameException e) {
    System.debug('Custom Exception: ' + e.getMessage());
} catch (Exception e) {
    System.debug('General Exception: ' + e.getMessage());
}
```

✅ Explanation:

- If Account name is "Test", throw your custom exception.

- `catch` handles this specific exception.

🌟 Example 7: Rethrowing an Exception

Scenario: Log the exception, then rethrow.

```
apex
try {
    Account acc = [SELECT Id FROM Account WHERE Name='Nonexistent' LIMIT 1];
    delete acc;
} catch (QueryException e) {
    System.debug('Logging error: ' + e.getMessage());
    throw e; // Rethrow the same exception
}
```

✅ Explanation:

- Exception is caught and logged.
- Then rethrown to propagate up the call stack.

🌟 Best Practices for Exception Handling in Apex

- ✅ **Use specific exception types** when possible (`DmlException`, `QueryException`).
- ✅ **Avoid swallowing exceptions silently.**
- ✅ **Always log or communicate errors** to admins or users.
- ✅ **Use `finally` for cleanup code.**
- ✅ **Use custom exceptions** to enforce business rules.
- ✅ **Avoid generic catch-all unless necessary**, as it can hide problems.

If you'd like, I can help you design more advanced examples or error-handling strategies for triggers, batches, or asynchronous Apex. Just let me know!